A practical dynamic-form pilot

Current solution proposal and local implementation, **11 October 2026**. AI-assisted assessment; customer integration and business acceptance remain pending.

Replace one email-and-spreadsheet request journey with configurable forms inside the existing React product. Reuse the product API for identity, permissions, data and workflow. Measure completion, missing-information returns and time to first review against a process-owner baseline; no unmeasured savings are claimed.

Delivered locally

- Session login, scrypt passwords, CSRF checks, mandatory password change, account reset and session revocation.

- Admin account/form management and server-enforced per-form create/view/review permissions.

- Restricted field editor, preview, immutable JSON publication and historical version pinning.

- Basic inputs, multi-select, nested objects, repeating groups/tables and nested AND/OR visibility with root/row/parent scopes.

- Draft save/reload, validated submission, real local file upload/preview/download, reviewer approval/rejection and audit.

- Strict draft upgrade preview/confirmation, exact snapshot restoration and optimistic revision conflicts.

All data is synthetic. Local sessions, JSON persistence and file bytes are real implementations, but are not the customer's SSO/database/file/workflow services. Files have policy/signature/access checks; antivirus, quarantine and retention are not implemented.

Decisions still needed

Confirm field limits, approval/resubmission policy, view/review scope, attachment storage/scanning/retention, translation ownership, publication approval, UAT metrics and named acceptance owner. Default rules are assessment assumptions.

Architecture and controlled change

React owns the product shell, selection, locale and network state. `DynamicForm` / `FormHandle` contains renderer integration. Legacy/basic schemas use Form.io; structured custom schemas use React fields with shared visibility/validation. The API owns authorization, validation, revision, status, immutable form history and request snapshots.

Publication and compatibility

An admin authors and previews a restricted declarative schema, saves its revision and publishes a new immutable version. New requests fetch the active definition. Existing requests fetch their pinned form ID/version. Arbitrary JavaScript and unapproved field kinds are rejected. Business/security publication approval remains a customer integration gate.

Supported field keys, labels, validators, nesting and conditions can change without a frontend release. New input kinds, operators, domain APIs or workflow policy still require implementation and regression testing. The adapter does not eliminate renderer/schema coupling.

Migration policy

The draft owner explicitly previews an upgrade. Removed/hidden values are listed for review; invalid supplied values remain editable. Every visible value and required field must validate against the new schema before confirmation. The server recomputes the candidate and checks revision, active target version and fingerprint before atomically writing a new revision/snapshot/audit.

Restore returns exact historical data, schema version and files while creating a new revision. Missing schemas/files or stale previews fail without mutation. Submitted/final requests cannot migrate. Ordinary save requests retain their current type/version contract.

Product integration

Use the existing verified session/SSO, scoped database policies, approved scanning/file service and reliable workflow dispatch. Database status/audit writes need transactions; workflow handoff needs idempotency. Local single-process JSON writes do not provide distributed transactions or operational retention guarantees.

Delivery and acceptance

Illustrative customer pilot estimate: four weeks; 38 base person-days plus eight contingency = 46 person-days. Re-estimate after confirming product contracts. This estimate is separate from the completed local assessment and does not claim the product integrations are delivered.

Accountable work packages

- Discovery, access rules, API and acceptance contract: 5 person-days.

- React integration, conditional UI, draft/reload and languages: 10 person-days.

- Product identity, persistence, files and workflow/status: 10 person-days.

- Verification, UAT support, fixes, rollout and handover: 8 person-days.

- Technical leadership, reviews and documentation: 5 person-days; contingency: 8.

Week 1 proves one authenticated product round trip and agrees scope. Week 2 completes the request/file journey. Week 3 integrates workflow and operational controls. Week 4 runs UAT, fixes, sign-off and rollout. Senior engineer owns boundaries and evidence; juniors take bounded list/status/translation tasks; backend owns policy; QA coordinates regression; business users own acceptance.

Risks and release gates

API mismatch requires an early contract spike and re-estimation. Unsafe configuration requires restricted authoring and business review. Unauthorized access or data loss blocks acceptance. Files require scanning/retention. Historical meaning requires immutable versions and explicit migrations. Bundle size, target-device performance and complete accessibility need measured product budgets. Sessions, rate limits and storage need distributed operational design before scaling.

UAT

Agree outcomes and entry conditions; use synthetic sandbox accounts and repeatable evidence. Verify conditions, invalid data, save/reload, grants, file access, upgrade/cancel/restore, concurrent saves and reviewer decisions. Exit requires all critical journeys passing, no open critical/high defects, accepted lower-severity issues with owners/dates, reviewed rollback/runbook and named business sign-off. Customer UAT has not occurred.

Evidence and handover

Local verification records cover **77 unit/API tests** and **11 Chromium scenarios**. `npm run check` enforces lint/format, tests and TypeScript/production build; every authored JS/TS file is at most 200 lines. `npm run test:e2e:headed` executes the visible browser suite with disposable accounts/data.

Schema-only release evidence

The integrated browser scenario publishes v2 with a new field, discovers it in the same requester document, preserves a v1 draft, corrects/confirms upgrade, submits and approves as a separate reviewer. It compares SHA-256 manifests before/after publication and hashes served assets. The unchanged document time origin and identical bundle demonstrate this supported JSON change needs no frontend build/deployment.

The HTML report contains the JSON evidence attachment and screenshots. See `docs/release-independent-schema.md` for reproduction and limits. Existing tests cover legacy/custom rules, permission bypasses, persistence/restarts, missing files/schemas, strict invalid candidates, restoration and conflict recovery.

CI and operational handoff

GitHub Actions installs locked development dependencies on Node 24/Ubuntu 24.04, runs checks and headless Chromium, and retains browser artifacts for 14 days. Hosted execution awaits a GitHub remote; configuration is not a claim of a successful remote run. Local production bundle and CSS warnings remain disclosed.

The source package includes lockfile, setup README, contracts, CI, tests, current architecture/decisions, delivery estimate, risk/UAT plans, verification, demo script and backup/recovery guidance. `docs/handoff.md` is the current runbook. No real customer data or credentials are included.

AI and candidate ownership

OpenAI Codex drafted implementation/tests/documents and executed the recorded checks. Assistant review is not independent candidate review. Candidate review/rehearsal, customer UAT, production integration and business acceptance remain pending. Personally inspect the code, rerun the demo and explain the trade-offs before submission.

Sources for the workflow: Playwright CI documentation (playwright.dev/docs/ci) and official GitHub Actions repositories. Renderer uses pinned Form.io packages; no hosted Form.io account or Enterprise capability is claimed. Verify procurement/licensing requirements separately if the customer selects additional offerings.